講義 1：ABAP 語法基礎

> 對應練習：ex01 | 答案程式：`ZR_TR01_SYNTAX_BASICS`

本講重點

- ABAP 程式怎麼建立、怎麼執行（SE38 / Executable Program）
- statement 以句點 `.` 結束，可自由跨行
- 兩種註解：`*`（整行）與 `"`（行內）
- 冒號 `:` 鏈式寫法（Chain Statement）
- 關鍵字不分大小寫與 Pretty Printer
- `WRITE` 輸出與 `/` 換行

1. ABAP 程式的執行環境

ABAP 程式不是存在你電腦裡的檔案，而是存在 SAP 系統資料庫中的「Repository 物件」，在應用伺服器上執行。本課程用到的工具：

交易代碼	用途
SE38	ABAP Editor：建立、修改、執行程式
SE80	Object Navigator：總覽套件下所有物件
SA38	只能「執行」程式（給一般使用者）

建立程式的固定步驟（本課程所有練習共用）：

1. SE38 → 輸入程式名（客戶自開發一律 `Z` 或 `Y` 開頭，課程用 `ZR_TRnn_姓名縮寫`）→ Create
2. 類型（Type）選 **Executable Program**（可直接 F8 執行的報表程式）
3. 套件（Package）填 `$TMP`（Local Object，不產生傳輸請求，適合練習）
4. 編輯器中撰寫 → `Ctrl+F2` 語法檢查 → `Ctrl+F3` 啟用（Activate）→ `F8` 執行

> 觀念：**存檔 ≠ 啟用**。程式有「作用中（active）」與「修訂中（inactive）」兩個版本，執行的是啟用過的版本。改完沒啟用，跑起來還是舊的——初學最容易困惑的地方。

每支 Executable Program 的第一個 statement 一定是：

```
REPORT zr_tr01_syntax_basics.
```

2. Statement 與句點

ABAP 的最小單位是 statement（陳述式），規則只有一條：**以句點 `.` 結束**。換行、縮排、多個空白都不影響意義，一個 statement 可以拆成很多行寫：

```
WRITE 'Hello ABAP!'.

* 跟上面那行完全等價：
```

```
WRITE
  'Hello ABAP!'
  .
```

因此 ABAP 沒有「一行一句」的概念——**編譯器只認句點**。這帶來兩個常見錯誤：

- 忘記句點：錯誤訊息往往指到「下一行」，因為編譯器把下一行當成同一句的延續。看到怪錯誤，先檢查上一行結尾。
- 句點打成逗號：鏈式寫法（見第 4 節）中間才用逗號，最後一定是句點。

3. 註解

寫法	規則	範例
`*` 整行註解	星號必須在**第 1 個字元**（行首），整行都是註解	`* 這是整行註解`
`"` 行內註解	從雙引號到行尾都是註解，可跟程式碼同行	`WRITE 'A'. " 說明文字`

```
* 整行註解：星號一定要貼在行首，縮排後的 * 會被當語法錯誤
WRITE 'Hello'.        " 行內註解：解釋這一行在做什麼
```

> 實務建議：註解寫「為什麼這樣做」，不要寫「這行在做什麼」（程式碼本身就看得出來的不用寫）。

4. 冒號鏈式寫法（Chain Statement）

多個 statement 開頭的關鍵字相同時，可以用冒號 `:` 合併，中間以逗號分隔、最後以句點收尾：

```
* 三句分開寫：
WRITE / 'A'.
WRITE / 'B'.
WRITE / 'C'.

* 鏈式寫法，跟上面完全等價：
WRITE: / 'A',
       / 'B',
       / 'C'.
```

展開規則：冒號前的部分會複製到每一段。所以：

```
DATA: gv_a TYPE i,
      gv_b TYPE i.
* 等價於
DATA gv_a TYPE i.
DATA gv_b TYPE i.
```

鏈式寫法在宣告區（`DATA:`、`TYPES:`）與輸出（`WRITE:`）最常用，是 ABAP 程式的「標準長相」，一定要會讀會寫。

5. 關鍵字不分大小寫

`write`、`Write`、`WRITE` 對編譯器完全相同（字面文字 `'...'` 裡的內容除外，那是資料，大小寫有差）。

```
write / 'lowercase 寫法'.
Write / 'Mixed Case 寫法'.
WRITE / 'UPPERCASE 寫法'.
```

團隊慣例靠 **Pretty Printer**（SE38 內按 `Shift+F1`）統一：可設定「關鍵字大寫、變數小寫」或全小寫等風格，寫完習慣按一下，全團隊格式一致。

6. WRITE 輸出

`WRITE` 把資料寫到「基本清單（basic list）」——執行畫面上看到的輸出。本階段所有練習都靠它觀察結果。

寫法	效果
`WRITE '文字'.`	接在目前輸出位置後面（同一行，間隔一格）
`WRITE / '文字'.`	先換行，再從新行開頭輸出
`WRITE: / 'A', 'B'.`	A 換行輸出，B 接在 A 後面

```
WRITE 'Hello ABAP!'.          " 第一行（清單起點）
WRITE '接在後面'.              " 不加 / ：同一行繼續
WRITE / '這是第二行'.          " 加 / ：換行
```

`WRITE` 後面可以接字面文字（`'...'`）、變數、系統欄位（如 `sy-datum` 今天日期），之後每一講都會用到；進階的位置與格式控制（欄位對齊、寬度）留到講義 12 列印排版。

> 輸出中文要注意：中文字在清單上每個佔兩格，一段中文字太長時結尾可能被截掉，而且不會有錯誤訊息。遇到時用 `WRITE /(60) '...'` 指定寬度即可，原因與排版規劃見講義 12 第 2.1 節。

7. 字面文字裡的單引號怎麼寫

字面文字 `'...'` 的分隔符號就是單引號本身，所以內容裡如果要放一個字面上的單引號，規則是**連續打兩個單引號 `''`**，編譯器會把它還原成一個單引號字元（跟 SQL 的逃脫規則一樣）：

```
WRITE 'It''s ok'.
```

輸出結果是 `It's ok`——中間的 `''` 不是兩個單引號字元，而是一個被逃脫過的單引號。

這條「雙寫逃脫」規則**只在單引號字面文字 `'...'` 成立**。如果改用反引號宣告 `string` 型別的字面文字，因為分隔符號換成了反引號，單引號完全不用逃脫，可以直接寫：

```
DATA lv_text TYPE string.
lv_text = `It's ok`.              " 反引號字串，單引號不用雙寫
```

`WRITE` **也可以直接接反引號字面文字**，不限定一定要用 `'...'`：

```
WRITE 'It''s Tom''s book.'.    " 兩個單引號都要雙寫，讀起來吃力
WRITE `It's Tom's book.`.      " 反引號完全不用逃脫，一眼看懂
```

兩句輸出完全一樣（`It's Tom's book.`）。內容裡單引號一多，改用反引號會明顯清楚很多——這是實務上常見的取捨，不是規定死一定要用哪一種。

記住這個原則：逃脫的對象永遠是「自己的分隔符號」——單引號字面文字要逃脫的是 `'`，反引號字面文字要逃脫的是 `` ` ``，就不會搞混。

> **補充：字串模板（String Template）`|...|` 新寫法**：7.40 之後還有第三種字面文字，分隔符號是 `|`，寫法 `WRITE |It's Tom's book.|.`，單引號同樣不用逃脫；規則跟著分隔符號走，所以要放字面上的 `|` 本身才需要逃脫：`WRITE |A\|B|.` 輸出 `A|B`。字串模板還能在 `{ }` 裡直接嵌入變數與運算式，講義 18 §1 的「新式寫法對照」會詳細教，這裡只需要知道它存在。

單引號 vs 反引號：不只是逃脫規則不同

兩者的**底層型別**不一樣，反引號（NW 7.00 起支援）就是為了補上型別 `c` 的限制而出現的：

比較項目	單引號 `'...'`	反引號 `` ` ... ` ``
型別	文字欄位（Text Field Literal，Type **c**）	字串（String Literal，Type **string**）
長度	固定長度	動態長度，隨內容增減
尾端空白	**被忽略**（自動裁掉）	**完整保留**
內含單引號 `'`	要寫兩個 `''`	直接寫 `'`
內含反引號 `` ` ``	直接寫 `` ` ``	要寫兩個

最容易踩到的差別是尾端空白：

```
DATA lv_s TYPE string.

lv_s = 'Hello   '.                  " 單引號：尾端空白被丟掉
WRITE: / strlen( lv_s ).            " 輸出 5

lv_s = `Hello   `.                  " 反引號：尾端空白保留
WRITE: / strlen( lv_s ).            " 輸出 8
```

這個差別在字串串接時會直接影響結果——想在兩段文字中間放一個空白，單引號寫法的空白會消失：

```
DATA: lv_1 TYPE string,
      lv_2 TYPE string.

lv_1 = 'SAP' && ' ' && 'ABAP'.      " 結果 SAPABAP（7 碼），空白不見了
lv_2 = `SAP` && ` ` && `ABAP`.      " 結果 SAP ABAP（8 碼），空白保留
```

> 這段已在 SAP 系統上實測。注意空白會消失的是用 `&&`（執行期串接運算子）的情況；兩個字面值之間用 `&`（編譯期字面值串接）時，`'SAP' & ' ' & 'ABAP'` 的空白**會**保留，結果是 `SAP ABAP`。

使用時機：

- 用單引號 `'...'`：固定長度的代碼值（如語言 `'E'`、狀態 `'A'`）；傳給只接受 `c` 型別參數的舊式 BAPI / Function Module。
- 用反引號 `` `...` ``：需要保留尾端空白時；字串內含大量單引號時（SQL 語法、JSON、HTML、英文縮寫）；需要動態組裝長字串時。

例如 SQL 字串，單引號要寫成 `'SELECT * FROM tab WHERE name = ''JOHN'''`，改用反引號就是 `` `SELECT * FROM tab WHERE name = 'JOHN'` ``，可讀性差很多。

8. 常見錯誤與陷阱

症狀	原因
錯誤訊息指在某行，那行看起來沒問題	上一行忘了句點
`*` 註解報語法錯誤	星號不在行首（前面有空白）
改了程式，執行結果沒變	只存檔沒啟用（Activate）
鏈式寫法（Chain Statement）報錯	中間用了句點、或最後用了逗號
字面文字裡想放單引號	ABAP 處理單引號逸出（Escape）的方式非常簡單：直接**連續寫兩個單引號** `''`，如 `'It''s ok'`。另外，ABAP（NW 7.00 以上）也支援用**反引號** `` ` `` 宣告 String Literal，內部的單引號完全不需要處理，如 `` `It's ok` ``（型別差異與使用時機見第 7 節）

9. 課堂練習

完成 ex01：用 WRITE、註解、鏈式寫法、大小寫混用、跨行 statement 各寫一段，驗證本講所有規則。